

数论专题

GESP五级考点详解

编号	知识块	知识点
1	初等数论	素数与合数、最大公约数与最小公倍数、同余与模运算、约数与倍数、质因数分解、奇偶性 欧几里得算法 唯一分解定理 素数表的埃氏筛法和线性筛法
2	算法复杂度估算方法	含多项式的算法复杂度 含对数的算法复杂度
3	C++高精度运算	C++数组模拟高精度加法、减法、乘法、除法
4	链表	单链表、双链表、循环链表的创建、插入、删除、遍历、查找的基本操作
5	二分算法	二分查找算法、二分答案算法（也称二分枚举法）
6	递归算法	递归算法的相关概念 递归算法的时间复杂度和空间复杂度 递归的优化策略
7	分治算法	归并排序算法、快速排序算法
8	贪心算法	贪心算法的相关概念、最优子结构

素数与合数

质数 (prime number) 又称素数，有无限个。定义为大于1的自然数中，除了1和它本身以外不再有其他因数。

合数 (composite number) 是与质数对应的概念，合数是指大于1的整数中，除了能被1和它本身整除外，还能被其他数整除的数。

用试除法验证数是否为素数

```
int isprime(int n) {  
    for (int i = 2; i < n; i++) {  
        if (n % i == 0) {  
            return 0;  
        }  
    }  
    return 1;  
}
```

$i \leq \sqrt{n}$

效率提升：埃氏筛和欧拉筛

最大公约数和最小公倍数

最大公约数也称为最大公因子，最大公因数，指两个或多个整数共有约数中最大的一个。

最小公倍数是指两个数或多个整数共有的倍数中除0以外最小的一个公倍数。

求最大公约数方法：辗转相除法（欧几里得算法）

用两个数中较大的数最为被除数，较小的数作为除数，进行相除，得到商和余数。接着将除数作为被除数，余数作为除数，继续重复相除，直到余数等于0，此时的除数就是两数的最大公约数。

求多个数的最大公约

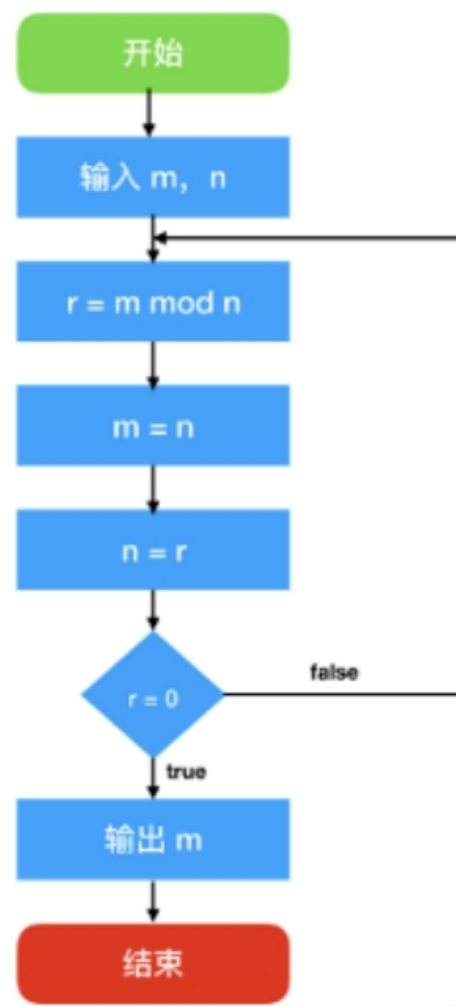
数： $\text{gcd}(a,b,c)=\text{gcd}(\text{gcd}(a,b),c)$

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 #include <iostream>
4 int gcd(int a, int b) {
5     if (b == 0) {
6         return a;
7     }
8     return gcd(b, a % b);
9 }
```

辗转相除法流程图

$$\begin{aligned} 22008 &\text{ mod } 655 = 393 \\ 655 &\text{ mod } 393 = 262 \\ 393 &\text{ mod } 262 = 131 \\ 262 &\text{ mod } 131 = 0 \\ 131 &\text{ 是最大公约数 } \mathbf{GCD} \end{aligned}$$

一般算法流程如下：



短除法和因式分解法

短除法 (求最大公约数)

把每个因数找出来，然后找出公因数中最大的。

$$\begin{array}{r|rr} 2 & 12 & 18 \\ \hline 3 & 6 & 9 \\ \hline & 2 & 3 \end{array}$$

因式分解法 (求最大公约数)

采取因式分解的形式来求最大公约数。

$$12 = 2 \times 3 \times 2$$

$$18 = 2 \times 3 \times 3$$

求最小公倍数方法：

- 1.如果两个数是倍数关系，则它们的最小公倍数就是较大的数；
- 2.相邻两个自然数的最小公倍数是它们的乘积；
- 3.最小公倍数 = 两数乘积 / 最大公约数

约数与倍数的表示方法：

设 a 是非零整数， b 是整数。如果存在一个整数 q ，使得 $b = a * q$ ，那么就说 b 可以被 a 整除，记作 $a|b$ ，则称 b 是 a 的倍数， a 是 b 约数（因子）。

例如：3|15，7|35

经典例题 01:

某政府机关内甲、乙两部门通过门户网站定期向社会发布消息，甲部门每隔 2 天、乙部门每隔 3 天有一个发布日，节假日无休。则甲、乙两部门在一个自然月内最多有几天同时为发布日（ ）

- A. 5 B. 2 C. 6 D. 3

经典例题 02:

企业某次培训的员工中有 369 名来自 A 部门，412 名来自 B 部门。现分批对所有人进行培训，要求每批人数相同且批次尽可能少。如果有且仅有一批培训对象同时包含来自 A 和 B 部门的员工，那么该批中有多少人来自 B 部门（ ）

- A. 14 B. 32 C. 57 D. 65

同余与模运算

模运算

设 a 为正整数, p 为整数, 一定存在等式 $a = k * p + r$,
其中 k, r 是整数, 且 $0 \leq r < p$, k 为商, r 为余数。

$a \% p$ ($a \bmod p$)表示 a 除以 p 的余数。

同余

若 a, b 两个整数的差 $a-b$ 能被某个自然数 m 所整除, 则称 a 就模 m 来说同余于 b , 或者说 a 和 b 关于模 m 同余。

$a \equiv b \pmod{m}$ 表示 a 和 b 模 m 同余, 即 a 和 b 除以 n 的余数相等。

例如 $45 \equiv 3 \pmod{7}$ 、 $79 \equiv 13 \pmod{11}$

◦ 余数定理

- $(a + b) \pmod{k} = (a \pmod{k} + b \pmod{k}) \pmod{k}$
- $(a \times b) \pmod{k} = (a \pmod{k} \times b \pmod{k}) \pmod{k}$
- $(a - b) \pmod{k} = (a \pmod{k} - b \pmod{k} + k) \pmod{k}$

质因数分解

唯一分解定理

任何一个大于1的整数 n 都可以分解成若干个素因数的连乘积，如果不计各个素因数的顺序，那么这种分解是唯一的。

即若 $n > 1$ ，则有 $n = p_1 * p_2 * p_3 \dots * p_m$ ，其中 $p_1 \leq p_2 \leq \dots \leq p_m$ 皆质数。

```
void fjjzys(int x){
    for(int i=2;i<=sqrt(x);i++){
        if(x%i==0)
            while(x%i==0){
                x/=i;
                if(x!=1)
                    cout<<i<<"*";
                else
                    cout<<i;
            }
    }
    if(x!=1) cout<<x;
}
```

奇偶性

我们使用%来判断一个数是奇数还是偶数，原理是如果除以2的余数为0为偶数，否则为奇数。

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int n;
7      cout << "输入一个整数: ";
8      cin >> n;
9      if (n % 2 == 0)
10         cout << n << " 为偶数。";
11     else
12         cout << n << " 为奇数。";
13     return 0;
14 }
```

*快速幂

- 核心思想：每一次运算都把指数折半，底数变其平方
- 每次的指数都折半可以把很大的指数不断减小，这样减少循环次数，但还能保持最终结果不变达到快速求幂次方的效果。例如： 4^{10} 第一次折半是 4^5 ，第二次折半是 4^2 ，第三次折半是 4^1 ，第四次折半是 4^0 。

计算 4^{12} :

$$4^{12} = 4 * 4 * 4 * 4 * 4 * 4 * 4 * 4 * 4 * 4 * 4 * 4$$

我们跟据指数折半底数变其平方来操作

$$(4 * 4)^{12/2} = 16^6 = (4 * 4) * (4 * 4) * (4 * 4) * (4 * 4) * (4 * 4) * (4 * 4) = 16 * 16 * 16 * 16 * 16 * 16$$

我们发现 4^{12} 变成了 16^6 ，循环次数从12变成了6，我们继续操作

$$(16 * 16)^{6/2} = 256^3 = (16 * 16) * (16 * 16) * (16 * 16) = 256 * 256 * 256$$

现在 16^6 次方变成了 256^3 ，循环次数从6变成了3，我们继续操作，但是会发现现在的指数是3，不能被二整除，那怎么办呢？**若指数是奇数那么指数减一再折半**。让3变成2再折半。

$$256 * (256 * 256)^{(3-1)/2} = 256 * 65536^1 = 256 * (256 * 256) = 256 * 65536$$

现在 256^3 变成了 $256 * 65536^1$ 因为现在的指数是1指数是奇数所以我们还是继续指数减一再折半的操作

$$256 * 65536 * 65536^{0/2} = 256 * 65536$$

快速幂

```
long long int quik_power(int base, int power)
{
    long long int result = 1;
    while (power > 0)           //指数大于0进行指数折半，底数变其平方的操作
    {
        if (power % 2 == 1)    //指数为奇数
        {
            power -= 1;        //指数减一
            power /= 2;        //指数折半
            result *= base;    //分离出当前项并累乘后保存
            base *= base;      //底数变其平方
        }
        else                   //指数为偶数
        {
            power /= 2;        //指数折半
            base *= base;      //底数变其平方
        }
    }
    return result;             //返回最终结果
}
```


优化快速幂

```
long long int quik_power(int base, int power)
{
    long long int result = 1;
    while (power > 0)                //指数大于0进行指数折半, 底数变其平方的操作
    {
        if (power & 1)              //指数为奇数, power & 1这相当于power % 2 == 1
            result *= base;         //分离出当前项并累乘后保存
        power >>= 1;                //指数折半, power >>= 1这相当于power /= 2;
        base *= base;               //底数变其平方
    }
    return result;                  //返回最终结果
}
```


数论高手挑战

P11961 [GESP202503 五级] 原根判断

提交答案

加入题单

复制题目

题目背景

复制 Markdown

展开

进入 IDE 模式

截止 2025 年 3 月，本题可能超出了 GESP 考纲范围。在该时间点下，原根是 NOI 大纲 8 级知识点（NOI 级），而相对简单的无需原根知识的做法中，使用的费马小定理与欧拉定理也属于 NOI 大纲 7 级知识点（提高级），且均未写明于 GESP 大纲中。需要注意，GESP 大纲和 NOI 大纲是不同的大纲。

若对题目中原根这一概念感兴趣，可以学习完成 [【模板】原根](#)。

题目描述

小 A 知道，对于质数 p 而言， p 的原根 g 是满足以下条件的正整数：

- $1 < g < p$;
- $g^{p-1} \bmod p = 1$;
- 对于任意 $1 \leq i < p - 1$ 均有 $g^i \bmod p \neq 1$ 。

其中 $a \bmod p$ 表示 a 除以 p 的余数。

小 A 现在有一个整数 a ，请你帮他判断 a 是不是 p 的原根。

输入格式

第一行，一个正整数 T ，表示测试数据组数。

每组测试数据包含一行，两个正整数 a, p 。

输出格式

对于每组测试数据，输出一行，如果 a 是 p 的原根则输出 `Yes`，否则输出 `No`。

输入输出样例

输入 #1	输出 #1
3	Yes
3 998244353	Yes
5 998244353	No
7 998244353	

说明/提示

数据范围

对于 40% 的测试点，保证 $3 \leq p \leq 10^3$ 。

对于所有测试点，保证 $1 \leq T \leq 20$, $3 \leq p \leq 10^9$, $1 < a < p$, p 为质数。

解题思路

1.数据量较大且计算复杂，使用快速幂加快次方计算，用取模降低数据大小。（定制一个KSM函数）

2.对于任意 $1 \leq i < p-1$ ，均有 $g^i \bmod p \neq 1$ 。
通过找规律思考——如果存在 $=1$ 的情况，该位置与 $gp-1$ 有何关系？（基于余数定理）

3.基于前两条思路，编写程序。需要注意的是：要去检验 $p-1$ 的因数位置是否符合条件（该条件不是一个充分必要条件）